

Oumelbenine Tayeb

Week 8 report

TCP Part 1 — Connections, Sequencing, and Flow Control TCP Part 2: Congestion Control and the Modern Picture

This report synthesizes the core concepts from Lectures 19 and 20, detailing the mechanisms of the Transmission Control Protocol (TCP), its evolution to handle network congestion, and the modern shift toward protocols like QUIC.

I. TCP Fundamentals: Connections and the Byte-Stream Model

TCP is the primary protocol used to achieve reliable, full-duplex communication over the unreliable IP layer. Unlike earlier models that numbered packets, TCP utilizes a byte-stream model, where sequence numbers reflect byte offsets rather than segment counts. This allows applications to treat data as a continuous stream, though it necessitates that applications impose their own message framing.

1. Connection Management

- **The Three-Way Handshake:** To establish a connection, TCP uses a three-message exchange (SYN, SYN-ACK, ACK) to synchronize Initial Sequence Numbers (ISNs) and negotiate options like Maximum Segment Size (MSS) and Window Scaling.
- **ISN Randomization:** ISNs are randomized rather than starting at zero to prevent stale segment rejection (where old data from a closed connection is accepted by a new one) and to enhance security against off-path sequence-number prediction attacks.
- **Graceful Teardown and TIME WAIT:** Connections close via a four-way FIN/ACK exchange. The "active closer" must enter a TIME WAIT state for $2 \times \text{MSL}$ (Maximum Segment Lifetime) to ensure the final ACK was received and to allow any remaining segments to expire in the network before a 5-tuple can be reused

2. Reliable Delivery and RTT Estimation

TCP achieves reliability through cumulative acknowledgments and retransmission timers.

- **RTT Estimation:** To set a robust Retransmission Timeout (RTO), TCP uses an Exponentially Weighted Moving Average (EWMA) to track the Smoothed RTT (SRTT). Because mean RTT alone is insufficient due to network variance, TCP also tracks RTTVAR (variance) to create a "confidence-interval" for the RTO.
- **Fast Retransmit:** To avoid long RTO stalls, TCP uses Fast Retransmit, where three duplicate ACKs serve as a signal that a segment was lost, triggering an immediate retransmission before the timer expires.

II. Managing Transmission: Flow vs. Congestion Control

A critical distinction in TCP is between flow control and congestion control. While flow control protects the receiver, congestion control protects the network itself.

1. Flow Control (rwnd)

Flow control is an explicit mechanism where the receiver advertises its available buffer space via the receive window (rwnd) field. The sender is constrained such that bytes in flight $\leq$ rwnd. Pathologies like Silly Window Syndrome and inefficient small-packet transmissions (addressed by Nagle's Algorithm) are managed to maintain protocol efficiency.

2. Congestion Control (cwnd)

Congestion control is implicit; the sender must infer network state from loss signals. The congestion window (cwnd) is the sender's estimate of network capacity. The effective window is the minimum of cwnd and rwnd.

TCP Reno uses the AIMD (Additive Increase, Multiplicative Decrease) algorithm:

- Slow Start: Begins with cwnd=1 and doubles every RTT to probe for bandwidth quickly.
- Congestion Avoidance: Once cwnd reaches the ssthresh (slow-start threshold), it grows linearly (+1 MSS per RTT).
- Multiplicative Decrease: On a timeout (severe loss), cwnd resets to 1; on three duplicate ACKs (mild loss), it halves.
- Fast Recovery: This phase allows TCP to skip slow start after a triple duplicate ACK, as those ACKs prove that some data is still reaching the receiver

The supplemental Python code demonstrates the Jacobson/Karns Algorithm; notably, during the 300ms sample spike, the RTO jumps significantly (to 441.2ms) because the variance weight ($4 \cdot \{RTTVAR\}$) dominates the calculation, showcasing TCP's conservative approach to retransmission.

III. Modern Enhancements and the Evolution to QUIC

As network speeds increased, the original TCP Reno algorithm faced limitations, particularly on High-Bandwidth-Delay-Product (BDP) paths.

1. High-Performance TCP

- Window Scaling: The standard 16-bit window field limits throughput to ~64KB. Window Scale allows windows up to 1GB, which is essential for 1Gb/s+ links.
- TCP CUBIC: The default in modern Linux, CUBIC uses a cubic function of wall-clock time rather than RTT-clocked linear growth. This allows for faster recovery on high-BDP links and improves RTT fairness between competing flows.

- ECN (Explicit Congestion Notification): ECN allows routers to mark packets as "Congestion Experienced" (CE) instead of dropping them, allowing the sender to reduce cwnd earlier and without losing goodput.

2. The Shift to QUIC

Despite its dominance, TCP suffers from structural issues like Head-of-Line (HOL) blocking, where one lost packet stalls an entire multiplexed stream (as in HTTP/2). QUIC, which runs over UDP, addresses these by:

- Independent Streams: Loss in one stream does not block others.
- Lower Latency: Combining transport and TLS handshakes to achieve 0-RTT or 1-RTT connection setup.
- Connection Migration: Using Connection IDs instead of 4-tuples to allow sessions to survive IP address changes (e.g., switching from WiFi to cellular).
- Anti-Ossification: Encrypting headers to prevent middleboxes from tampering with the protocol.

Conclusion

The progression from basic reliable delivery (Lecture 19) to complex congestion management and modern encrypted protocols (Lecture 20) highlights the ongoing challenge of balancing efficiency, fairness, and latency in the global internet architecture

Result from python code:

Sample: 100ms | SRTT: 100.0 | RTTVAR: 50.0 | RTO: 300.0ms
Sample: 105ms | SRTT: 100.6 | RTTVAR: 38.8 | RTO: 255.6ms
Sample: 300ms | SRTT: 125.5 | RTTVAR: 78.9 | RTO: 441.2ms
Sample: 110ms | SRTT: 123.6 | RTTVAR: 63.1 | RTO: 375.9ms
Sample: 100ms | SRTT: 120.7 | RTTVAR: 53.2 | RTO: 333.5ms